

DevStreaks — Interview Q&A

Level: Beginner → Medium | **Questions:** 30 **Stack:** React Native, Expo SDK 54, TypeScript, Expo Router, Supabase (Auth + PostgreSQL), Kotlin (Android widgets)
Repo: <https://github.com/ermadhav/DevStreaks> | **Live:** <https://www.devstreaks.site>

Important: These answers are based on your README and resume, not your source code. Change any answer so it matches what you actually built. DevStreaks is your strongest project (shipped product, real users, OAuth, backend, native code), so expect the interviewer to spend the most time here.

Table of contents

1. [Basics and tooling](#)
 2. [Architecture and state](#)
 3. [Data sources and streak logic](#)
 4. [Auth, security and backend](#)
 5. [Notifications, widgets and UI](#)
 6. [Shipping and reflection](#)
-

1. Basics and tooling

Q1. Explain DevStreaks in 30 seconds.

A: DevStreaks is a cross-platform mobile app that shows a developer's GitHub and LeetCode activity in one dashboard: current and longest streaks, contribution heatmaps, and Easy/Medium/Hard breakdowns. It has reminders, home screen widgets, dark/light themes and a shareable developer card. It's live at devstreaks.site and has 100+ downloads.

Q2. Why did you build it?

A: I track my own coding consistency on both GitHub and LeetCode, and I wanted one place that showed both instead of switching between apps. Building it for myself gave me real requirements, and I then shipped it so other developers could use it.

Q3. What tech stack did you use?

A:

- **Framework:** React Native + Expo SDK 54
- **Language:** TypeScript
- **Navigation:** Expo Router (file-based)
- **State:** React Context + AsyncStorage
- **Backend:** Supabase (Auth + PostgreSQL)
- **Notifications:** `expo-notifications` + `expo-background-fetch`
- **Widgets:** custom Android `AppWidgetProvider` in Kotlin

- **Secure storage:** `expo-secure-store`

Q4. Why React Native instead of native Android/iOS or Flutter?

A: One TypeScript codebase for both platforms, and I already knew React from my web projects, so I could ship faster. Native would mean two codebases. Flutter would have meant learning Dart. Where I needed native features, like widgets, I wrote that part in Kotlin.

Q5. What is Expo, and what does it give you?

A: Expo is a framework and toolchain on top of React Native. It provides ready-made modules (notifications, secure storage, linking), a dev workflow, and **EAS Build** to create Android and iOS builds in the cloud without setting up native tooling locally.

Q6. What is a development build (`--dev-client`), and why not just use Expo Go?

A: Expo Go only includes a fixed set of native modules. My app uses custom native code (the Kotlin widgets), which Expo Go can't load. A dev client is a custom build of my app that includes those native parts but keeps fast refresh for development.

Q7. How does Expo Router work?

A: It's file-based routing, like Next.js: every file inside `app/` becomes a screen. `_layout.tsx` files define shared layouts, and a folder like `(tabs)` defines the bottom tab navigator. My `_layout.tsx` also acts as an auth guard, redirecting unauthenticated users to login.

2. Architecture and state

Q8. Explain your folder structure.

A: `app/` holds screens, `components/` holds reusable UI (StreakCard, Heatmap, ActivityChart), `hooks/` holds data logic (`useGithubStreak`, `useLeetCodeStreak`), `services/` holds OAuth, notifications and widget bridging, `context/` holds Auth and Theme state, `constants/` holds design tokens, and `lib/` holds the Supabase client and schema. Screens stay thin and logic lives in hooks and services.

Q9. Why React Context and not Redux?

A: My global state is small: the logged-in user and the theme. Context is built in and enough for that. Redux would add boilerplate without much benefit. If the state grew complex or needed heavy cross-screen updates, I'd consider Zustand or Redux Toolkit.

Q10. What is AsyncStorage, and what are its limits?

A: It's a simple key-value store that persists data on the device. It's asynchronous, unencrypted, and only stores strings (so objects need `JSON.stringify`). It's good for cached stats and settings but not for secrets or large datasets.

Q11. What are custom hooks, and why did you use them?

A: A custom hook is a function that uses other hooks to package reusable logic.

`useGithubStreak` handles fetching, caching, loading and error state for GitHub data, so screens just call the hook and render. That keeps UI code clean and logic testable.

Q12. How does caching work in your hooks?

A: *(Adjust to your implementation.)* On load, the hook first shows the cached data from AsyncStorage immediately, then fetches fresh data in the background and updates both state and cache. If the network fails, the user still sees the last known stats instead of an empty screen.

Q13. What is TypeScript, and how did it help?

A: TypeScript adds static types to JavaScript. In DevStreaks, I define types for API responses and props, so mistakes like a wrong field name show up at compile time. It made refactoring safer, especially as the project grew.

3. Data sources and streak logic

Q14. Where does the data come from?

A: GitHub contribution data comes from a public GitHub contributions API. LeetCode stats come through a community LeetCode API proxy. If the user connects with GitHub OAuth, the app can also include private contributions.

Q15. Why does OAuth matter for GitHub data?

A: Public endpoints only see public activity. GitHub's official contribution data, including private contributions, is available through its authenticated API (the GraphQL API's contributions data), which needs a token with the user's permission. OAuth lets the user grant that access without giving me their password.

Q16. What is GraphQL, and how does it differ from REST?

A: REST has many endpoints, each returning a fixed structure. GraphQL has a single endpoint where the client sends a query describing exactly which fields it wants, so there's less over-fetching and fewer round trips. GitHub offers both a REST and a GraphQL API.

Q17. What if the third-party APIs go down or change?

A: That's a real risk since I don't control them. Mitigations: cache the last good response, fall back gracefully with clear error messages, and show the user stale data instead of nothing. A stronger fix would be my own small backend that fetches, caches and normalizes data so the app depends on one stable interface.

Q18. How do you calculate a streak?

A: Take the daily contribution counts sorted by date. Walk backward from today and count consecutive days with a count above zero. Today gets a grace period: if there's no activity

yet today, start counting from yesterday, because today isn't over. The longest streak is a single pass keeping a running count and a maximum.

ts

```
function currentStreak(days: { date: string; count: number }[]) {
  // days sorted ascending by date; last item is today
  let i = days.length - 1;
  if (days[i]?.count === 0) i--; // today not over yet
  let streak = 0;
  while (i >= 0 && days[i].count > 0) {
    streak++;
    i--;
  }
  return streak;
}
```

Q19. What edge cases can break streak calculation?

A: Time zones (GitHub counts days in UTC, but the user lives in a local zone, so "today" can differ), missing days in the data, and days spanning midnight. I normalize dates to one convention and document it.

Q20. How is the heatmap rendered?

A: As a grid of small colored squares, one per day, arranged in weeks like GitHub's graph. Color intensity depends on the day's count, bucketed into levels. I render it from a fixed-size array, and use horizontal scrolling for long ranges.

4. Auth, security and backend

Q21. Explain the GitHub OAuth flow in your app.

A:

1. The app opens GitHub's authorization page in the browser.
2. The user approves access.
3. GitHub redirects to my app using a custom URL scheme (`devstreaks://oauth`) with a temporary code.
4. The app exchanges the code for an access token and stores it securely.

Q22. What is deep linking / a custom URL scheme?

A: It's a way for a URL to open a specific app. Registering `devstreaks://` tells the OS to open my app whenever that scheme is used. That's how the OAuth redirect returns the user from the browser back into the app.

Q23. Why `expo-secure-store` instead of `AsyncStorage` for tokens?

A: `AsyncStorage` is unencrypted plain storage. `expo-secure-store` uses the Android Keystore and iOS Keychain, which are designed for secrets. Tokens go in `SecureStore`; theme and cached stats can go in `AsyncStorage`.

Q24. What does Supabase provide?

A: Supabase is an open-source backend platform. I use its **Auth** (signup/login, sessions) and its hosted **PostgreSQL** database for profile sync. Its client library lets the app talk to the database directly.

Q25. What is Row Level Security (RLS), and why did you enable it?

A: RLS is a PostgreSQL feature where the database enforces row-level permissions through policies, for example "a user can only read and write rows where `user_id = auth.uid()`". It matters because the Supabase anon key ships inside the app, so anyone could extract it. RLS makes sure that key alone can't read other users' data.

Q26. Why did you add a local-only fallback?

A: Not every user wants an account, and the app shouldn't stop working if Supabase is unavailable. In local mode, everything stays on the device. Cloud sync is optional. That lowers the barrier for new users and improves privacy.

Q27. How do you keep secrets out of your repo?

A: Environment variables in a `.env` file that's in `.gitignore`, with a `.env.example` showing the required names. Note that `EXPO_PUBLIC_` variables are bundled into the app and visible to users, so only public values (client ID, Supabase anon key) go there. Real secrets, like an OAuth client secret, must live on a server.

5. Notifications, widgets and UI

Q28. How do the "smart" streak reminders work?

A: The app schedules a daily reminder with `expo-notifications`. A background task (`expo-background-fetch`) checks whether the user has already contributed today, and only shows the reminder if they haven't. That avoids nagging people who already kept their streak. Background execution on mobile is controlled by the OS, so timing isn't exact.

Q29. How did you build the Android home screen widgets?

A: React Native can't render home screen widgets, so I wrote a native `AppWidgetProvider` in Kotlin. On the JavaScript side, `WidgetService.ts` sends the latest heatmap data across a native bridge into shared storage, and the widget reads and draws it. It was the hardest part because it meant working outside the React Native layer and debugging Android-specific behavior.

Q30. How did you handle dark/light themes and animations?

A: A `ThemeContext` stores dark, light or system mode, and components read colors from a central `Theme.ts` through a `useTheme()` hook, so no screen hardcodes colors. Animations, like the share card flip, use `react-native-reanimated`, which runs on the UI thread and stays smooth even if the JS thread is busy.

6. Shipping and reflection

Have short, honest answers ready for these common follow-ups:

- **How did you release it?** EAS Build to create the app, a store listing, a privacy policy and terms page (the repo has `policy.tsx` and `terms.tsx`, and stores require this), a website at devstreaks.site, and monthly updates.
- **How did you get 100+ downloads?** Say what you really did (posts on X/LinkedIn, sharing with friends and communities), and that real users found bugs you never would have found yourself.
- **Biggest challenge?** Good options: the Kotlin widget bridge, OAuth redirect handling, background tasks behaving differently across Android versions, or streak logic across time zones. Explain the problem, what you tried, and what fixed it.
- **What would you improve?** Unit tests for hooks and services (the README lists this as an open idea), a small backend in front of the third-party APIs, iOS widgets, and achievements or friend leaderboards.
- **Which SQL do you know?** DevStreaks uses PostgreSQL through Supabase with RLS policies. That's your concrete SQL example if they ask about the SQL/MySQL on your resume.